

LLQM: Timeline-First Verification System

A LangGraph-based investigation agent that builds chronological timelines of news events and rumors, then produces confidence-tiered verdicts with full citation trails.

1. Quick Access and Running LLQM

If you're new to LLQM, start here: this is the repository, the main components, and the fastest way to run the system.

Code Location

Repository: <https://github.com/lucaslucaslu/LLQM>

Key components: - [src/llqm/service/verification_service.py](#): Core verification logic - [src/llqm/utlis/source_registry.py](#): Source trust scoring - [src/llqm/service/investigation_service.py](#): Claim extraction

Running the System

Environment setup:

Copy `.env.example` to `.env` in the project root, then fill in the required values:

```
copy .env.example .env
```

```
OPENAI_API_KEY=your_key_here
OPENAI_MODEL=gpt-5.4-mini
SERPER_API_KEY=your_key_here # optional, improves live search quality
```

OPENAI_API_KEY is required for LLM-assisted extraction/verification. SERPER_API_KEY is optional; without it, retrieval falls back to DuckDuckGo.

Interactive Web UI:

```
llqm-ui
```

Features: Live timeline building, claim drill-down, source authority visualization, verdict generation.

Command-line Interface:

```
llqm "your question here" --live
llqm --url https://news.article --live
```

Batch Testing:

```
python tests/test_disinformation.py
```

2. System Overview

LLQM (■■■■, "source and path") combines timeline reconstruction with source trust scoring to investigate claims and produce confidence-weighted verdicts backed by complete evidence trails.

Pipeline Architecture

```
Plan → Retrieve → Extract → Verify → Synthesize
↓ ↓ ↓ ↓ ↓
Query Web Search Claims Scoring Verdict +
Expansion to Docs + Events + Trust Confidence
```

Key Output: `InvestigationResult`

Each investigation produces a structured result containing:

Component	Description
Verdict	supported, likely_false, or unverified
Confidence	0.0–1.0 score derived from source trust + evidence corroboration + contradiction detection
Citations	Complete evidence trail with URL, source trust score, excerpt, and publication date
Uncertainty Notes	Identified gaps or conflicting evidence

3. Source Trust Scoring

LLQM assigns trustworthiness to sources through four layered mechanisms, combining editorial reputation, institutional signals, page-level metadata, and LLM classification.

Editorial Source Registry

A curated database of ~40 major outlets with pre-assigned trust scores:

Source Type	Examples	Trust Score
Wire services	Reuters, AP, AFP	0.90–0.92
Major newspapers	NYT, Washington Post, Guardian	0.85–0.86
Fact-checkers	Snopes, FactCheck.org	0.82–0.84
Social media	Twitter, Reddit	0.25–0.40
Blogs	Medium, Substack	0.30–0.40

Rationale: Editorial outlets maintain verification standards and face reputational consequences; social platforms and blogs do not.

Institutional Domain Signals

Domain-level trust is assessed by top-level domain:

Domain	Trust Score	Signal
.gov, .mil	0.88	Government institutions
.edu, .ac.uk	0.82	Academic institutions
.int	0.80	International organizations
.com, .org	—	Default to other signals

Rationale: Institutional domains face reputational and legal accountability.

Page-Level Signals

Structured data and metadata provide verification signals:

Signal	Trust Boost	Requirement
Schema.org NewsArticle type	+0.08	Professional publishing platform
Author attribution	+0.05	Accountability

Published timestamp	+0.04	Transparency
Corrections/ethics policy	+0.06	Editorial standards

Rationale: Professional outlets publish structured metadata; hoax sites do not.

LLM Classification Fallback

For unknown domains, the system uses LLM analysis:

```
{
  "source_type": "wire_agency|newspaper|blogger|forum|unknown",
  "trust_score": 0.0-1.0,
  "reason": "Justification based on domain patterns and context"
}
```

Rationale: LLM can recognize domain naming patterns and contextual clues.

4. Claim Scoring and Confidence Calculation

Claims are scored by evaluating corroboration across sources, source trust levels, and contradiction signals.

Scoring Formula

```
support_component = min(1.0, claim.support_count / 3.0)
contradiction_penalty = min(0.7, claim.contradiction_count * 0.25)
avg_trust = mean([e.trust_score for e in claim.evidence])

confidence = max(0.0, min(1.0,
  0.6 * support_component +
  0.4 * avg_trust -
  contradiction_penalty
))
```

Component Breakdown

Component	Weight	Meaning
Support Component	60%	Number of independent sources (normalized to 3)
Trust Component	40%	Average trust score of evidence sources
Contradiction Penalty	Variable	Reduction for negation or refutation signals

Scoring Logic

- **High corroboration + high trust** → High confidence (0.7–1.0)
- **Low corroboration + low trust** → Low confidence (0.0–0.3)
- **High support + explicit contradiction** → Confidence penalty applied
- **Single high-trust source** → Moderate confidence pending corroboration

5. The Critical Gap: Source Trust vs. Truth

The Core Problem

Source trust scoring is necessary but insufficient for truth-detection. It conflates source reputation with claim accuracy, creating systematic vulnerabilities when adversaries understand this gap.

The fundamental challenge: Source trust measures *verification intensity*, not *claim accuracy*. Reuters' 0.92 score reflects institutional rigor, not the truth value of every claim Reuters publishes.

Three Failure Modes

Failure 1: Coordinated Misinformation at Scale

Scenario: A coordinated disinformation campaign places identical false claims across 5+ high-trust outlets (AP, Reuters, BBC) simultaneously via shared wire relay or coordinated deception.

System sees	Reality
5 independent sources, confidence ≈ 0.85	All sources republished the same unverified claim
Verdict: Supported	Should be: Unverified or false

Current limitation: System has no way to detect republication chains. It treats independent domain names as independent verification.

Failure 2: Editorial Trust Decay Under Pressure

Scenario: A normally-reputable outlet (trust = 0.85) publishes a breaking-news claim without fact-checking. The claim is later quietly retracted but continues spreading through aggregators.

Timeline	System Assessment
Day 1: Reuters publishes claim (0.85 trust)	Confidence: 0.65
Day 5: Snopes publishes debunking	Penalty applied, verdict: Unverified
Reality	Should have been: Likely false (retraction source > original)

Current limitation: Contradiction detection only looks for explicit negation phrases in excerpts, not timing or authority weighting of retractions.

Failure 3: The Reputational Free Lunch

Scenario: A fringe outlet publishes a true but obscure scientific finding. Reuters republishes it 2 weeks later; system now sees two sources (fringe 0.30, Reuters 0.92), avg_trust = 0.61.

The problem: Reuters didn't independently verify—they republished. The system rewards the second publication as *independent verification* rather than *amplification*.

Current limitation: No detection of citation chains. The system can't distinguish "Reuters did their own investigation" from "Reuters copied from fringe outlet."

Why Source Trust Fails at Scale

Source trust is a proxy for institutional reliability, not claim accuracy:

- **Reuters** (0.92): large professional newsroom and editorial process → lower likelihood of publishing easily falsifiable lies
- **Reddit user** (0.30): Might discover truth through primary research but no institutional verification
- **Substack bot** (0.40): Might publish propaganda but accidentally be correct

When adversaries know this scoring model, they exploit it:

- 1. Plant false claim at high-trust outlet (via compromised journalist, misleading press release, or omitted context)
- 2. Wait for propagation through lower-tier outlets and social media
- 3. System marks claim "supported" because multiple sources now report it
- 4. Damage done; retraction (if any) arrives later and carries lower confidence weight

6. Current Mitigations (Incomplete)

The system attempts several defenses, each with known limitations:

Mitigation	Implementation	Limitation
Duplicate detection	Flag identical phrasing across 5+ outlets	Paraphrased versions slip through; doesn't scale well
Contradiction detection	Search for negation phrases in excerpts	Misses subtlety; doesn't weight timing or authority
Rumor-origin tracing	Identify earliest source of claim	Doesn't conclude "origin probably unverified"; still scores amplification as support
Confidence decay rules	Downgrade old or unspecified claims	Ad-hoc; doesn't address structural problem

Key gap: Mitigations are *defensive patches*, not architectural solutions. They reduce false positives but don't eliminate the core vulnerability.

7. Direction Forward: Oracle-Separated Verification

Separate Claim Producer from Verifier

The core insight: The same LLM that extracts claims from high-trust sources also grades whether to believe them. This creates implicit bias—if Reuters says it, extraction respects that authority; grading inherits that respect.

Solution architecture:

Claim Producer (fast, high-throughput): ■ Extracts candidate claims ■ Produces initial verdict ■ Assigns provisional confidence
Independent Oracle Verifier: ■ Receives only claim text (not source ID) ■ Retrieves independent evidence set ■ Re-grades against fresh sources ■ Outputs confidence blind to producer's score

Outcome: A significant gap between producer and oracle confidence triggers investigation mode—something is systematically wrong.

Build Regression Corpora by Claim Class

Different claim types require different verification approaches. Build golden sets (manually verified by domain experts) and refute sets (adversarial variants) for each class:

Price Claims (freshness decay: days)

Component	Details
-----------	---------

Test	"Is Product X sold for \$Y?"
Corpus	Price snapshots, retail databases, recent quotes
Adversarial variants	\$Y-\$50, \$Y+\$50, \$Y from 1 year ago

Materials Claims (freshness decay: years)

Component	Details
Test	"Does Product X contain Material Y?"
Corpus	Ingredient lists, regulatory filings, lab tests
Adversarial variants	Similar products, obsolete formulations, conflicting claims

Event Claims (freshness decay: permanent)

Component	Details
Test	"Did Event X happen on Date Y?"
Corpus	News archives, primary documents, timelines
Adversarial variants	Misquotes, reframed events, out-of-context excerpts

Track verifier accuracy per class and maintain a scoring dashboard.

Upgrade Contradiction Analysis

Replace shallow negation detection with structured analysis:

```
# Current approach: Light penalty
contradiction_penalty = min(0.7, contradiction_count * 0.25)

# Proposed approach: Structured analysis
if contradiction_found:
    - Authority weight: Is contradiction from higher-trust source?
    - Explicitness: Explicit negation ("X is false") vs. implicit (missing from authoritative reference)?
    - Timing: Retraction same day vs. retraction 6 months later?
    - Domain expertise: Contradiction from domain expert or general reporter?

if (authority_weight > claim_source_weight AND
    explicit AND
    recent):
    verdict ← "likely_false"
else:
    verdict ← "unverified"
```

Trace Provenance Chains

Track the full citation chain, not just the origin:

```
Outlet Z claims: "Study shows X"
↓ cites
Outlet Y claims: "Study shows X"
↓ cites
Outlet X (primary): "Our study shows X"
↓ cites
Preprint on arxiv

Classification:
• X = primary (conducted work)
• Y = secondary (cited X)
• Z = tertiary (cited Y)
```

Trust application:

- X gets full trust for original finding
- Y gets reduced trust (dependent on X)
- Z gets further reduced (dependent on Y)

Without this: The system treats Z's republication of Y's summary of X's claim as three independent data points. **With this:** Only X's claim gets full trust; later citations receive appropriately discounted confidence.

8. Real-World Testing Results

Test Dataset

Metric	Value
Cases	10 labeled disinformation campaigns
Dataset	tests/disinformation_results.json
Runner	tests/test_disinformation.py
Settings	max_iterations=2, live retriever enabled, LLM enabled
Date run	2026-06-21

Results Summary

Overall accuracy: 9/10 strict (90%) If `unverified` is counted as a safe non-supporting outcome for legacy disputed claims, operationally this is 10/10 non-supporting or correct.

Status legend: ✓ = exact label match, ■ = safe non-supporting mismatch.

Query	Ground Truth	System Verdict	Confidence	Status
Do vaccines cause autism?	FALSE	likely_false	0.99	✓
Does 5G spread COVID-19?	FALSE	likely_false	0.99	✓
Was the Moon landing a hoax?	FALSE	likely_false	0.98	✓
Is the Earth flat?	FALSE	likely_false	0.99	✓
Was the 2020 election stolen?	FALSE	likely_false	0.99	✓
Is Pizzagate real?	FALSE	likely_false	0.99	✓
Did Biden fall during campaign?	TRUE	supported	0.99	✓
Did hydroxychloroquine cure COVID?	FALSE	likely_false	0.98	✓
Did UFO crash at Roswell?	FALSE	unverified	0.93	■
Is JFK still alive?	FALSE	likely_false	0.99	✓

What Worked

The system performed strongly on this small historical disinformation set, especially for claims where:

1. Fact-checkers and major outlets published explicit debunking (Snopes, FactCheck.org, Reuters)
2. Contradictions were clear and high-trust (health authorities, government records)
3. Claim had extensive coverage in searchable news archives

Example: Pizzagate - Extracted claims: 5 key claims from 3+ independent sources - Verdict: `likely_false` (0.99 confidence) - Evidence chain: Original false claim → social media amplification → news coverage → fact-checker rebuttal - Key success: System correctly weighted fact-checker sources

above amplification sources

Edge Cases: Recent True Events and Legacy Conspiracy Claims

Biden Fall (Actual: TRUE, System: SUPPORTED, confidence: 0.99)

The system had to distinguish: - 2021 boarding mishap (real but old) - 2023–2024 campaign falls (real and recent)

Result: System correctly handled temporal filtering and returned appropriate confidence.

Roswell (Expected: FALSE, System: UNVERIFIED, confidence: 0.93)

This is a conservative miss: the system did not support the UFO claim, but also did not fully debunk it.

Result: Contradiction evidence for Roswell appears weaker/less explicit in retrieved sources than modern, fact-check-rich narratives.

Critical Limitation

Near-perfect performance on solved cases can mask a structural problem: These are historical disinformation claims with public debunking.

The system essentially traces the debunking path journalists created. This works until it doesn't.

Real Vulnerability: The Coordination Window

Consider this scenario (not in test set):

- 1. **Day 1:** Coordinated campaign plants false claim at Reuters, AP, BBC via shared wire relay
- 2. **Day 1–3:** Before fact-checkers respond, high-trust sources amplify the claim
- 3. **You run LLQM during window:** System sees 3 high-trust sources → confidence ≈ 0.85 → verdict: supported
- 4. **Day 5:** Fact-checkers publish rebuttals; damage already done

System behavior: - Sees: 3 independent domain names - Should detect: All three republished identical claim - Currently does: Treats republication as corroboration

This is the oracle-separation problem in action.

Accuracy by Claim Type

Claim Type	Strict Accuracy	Notes
Historic false claims (n=3)	100% (3/3)	50+ years of archives, strong debunking coverage
Recent health disinformation (n=3)	100% (3/3)	Active fact-checker coverage and medical consensus
Political/event claims (n=3)	100% (3/3)	Heavily covered, clear contradiction trails
Legacy conspiracy claims (n=1)	0% (0/1) strict	Roswell returned unverified (safe, but not a hard debunk)
Recent true events (n=1)	100% (1/1)	Temporal filtering worked correctly in this run

Potential break scenarios (not directly tested in this 10-case set): - True claims that *sound* false (e.g., "government tested vaccine on X population") - False claims *coordinated across high-trust sources* before debunking is available

Note: This dataset is intentionally small (10 cases) and should be treated as a directional signal, not a production-grade benchmark.

9. Key Takeaways

What This System Gets Right

- **Comprehensive timeline building:** Chronological event reconstruction from unstructured documents
- **Multi-layered source trust:** Combines editorial reputation, domain structure, page metadata, and LLM classification
- **Citation trails:** Every verdict is backed by retrievable, traceable evidence
- **Confidence scoring:** Quantified by corroboration, source trust, and contradiction signals

What Remains Open

The system solves *retrieval and ranking* effectively but cannot defend against:

- **Coordinated high-trust source amplification:** When multiple outlets republish the same unverified claim
- **Editorial trust decay:** When normally-reliable sources publish unverified breaking news
- **Provenance obfuscation:** When secondary sources appear as independent verification
- **Timing attacks:** When false claims propagate before fact-checkers respond

The path forward requires oracle-separated architecture: An independent verifier that re-checks claims blind to source identity, compared against regression corpora designed for specific claim types, with structured contradiction analysis and full provenance tracing.

10. Paid Work Trial Plan (One Week)

Product.ai evaluates the paid trial on how quickly I can ground myself in the real environment, write a clear verification spec before building, verify agent outputs rigorously, and assess my own work honestly. This plan is optimized for that bar.

10.1 Trial Objective

Ship one measurable improvement to verification quality in one live loop, while producing artifacts the team can audit quickly:

1. A short system map of current verification flow
2. A failure taxonomy grounded in Product.ai data
3. One implemented verifier or gate improvement
4. Before/after metrics and a candid self-assessment

10.2 Day-by-Day Execution

Day 1: Grounding and System Map - Ingest Product.ai data model: claims, evidence schema, confidence tiers, freshness logic, and citation structure - Trace one production-like agent loop end-to-end - Deliverable: 1-2 page verification flow map with observed risks and open questions

Day 2: Baseline and Failure Discovery - Build a small but representative evaluation slice from real claim-sets - Compute baseline metrics (precision/recall by claim class, contradiction catch-rate, stale-claim rate) - Deliverable: baseline table plus top failure patterns ranked by frequency and impact

Day 3: Spec Before Build - Select the single highest-impact failure class - Write a concise verification spec: acceptance criteria, guardrail behavior, fallback rules, and rollback plan - Deliverable: implementation-ready spec reviewed with stakeholders

Day 4: Implement One Improvement - Add one scoped verifier enhancement (for example: stronger contradiction weighting or provenance discounting) - Run in shadow mode or reversible mode first - Deliverable: working code path and run logs

Day 5: Evaluate, Report, and Self-Assessment - Run before/after comparison on the same evaluation slice - Quantify quality movement and token-cost impact - Deliverable: trial report with wins, misses, unknowns, and next-step recommendations

10.3 What I Will Not Do During Trial Week

- I will not attempt a broad platform rewrite.
- I will not force LLQM patterns onto Product.ai without evidence.
- I will not optimize for an impressive demo over measurable verification gain.

10.4 Success Criteria for the Week

1. I can explain Product.ai's current verification behavior with evidence.
2. I ship one improvement that reduces a high-impact failure mode.
3. The improvement is measurable, reversible, and documented.
4. I provide an honest self-assessment of what worked, what failed, and what I would do next.

10.5 Guiding Principle

Learn the real system first, write the spec before the build, ship one high-leverage improvement, and prove the result with data.